
TÉCNICAS DE APRENDIZADO DE MÁQUINA

Bacharelado em Sistemas da Informação

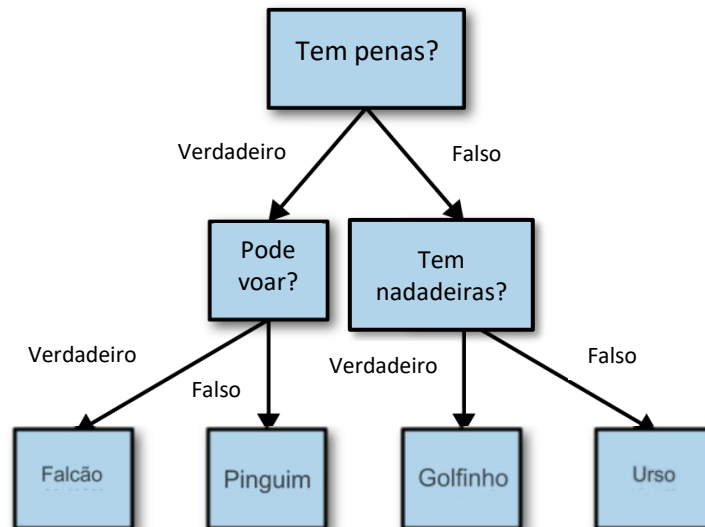
Prof. Marco André Abud Kappel

Aula 11 – Regressão com Árvores de Decisão

Regressão com Árvores de Decisão

- **Problemas de Regressão**

- Em problemas de regressão, nosso objetivo é estimar o **valor numérico** de uma variável dependente, dado um conjunto de variáveis independentes.
- Até aqui, utilizamos a técnica das Árvores de Decisão para resolver apenas **problemas de classificação**.

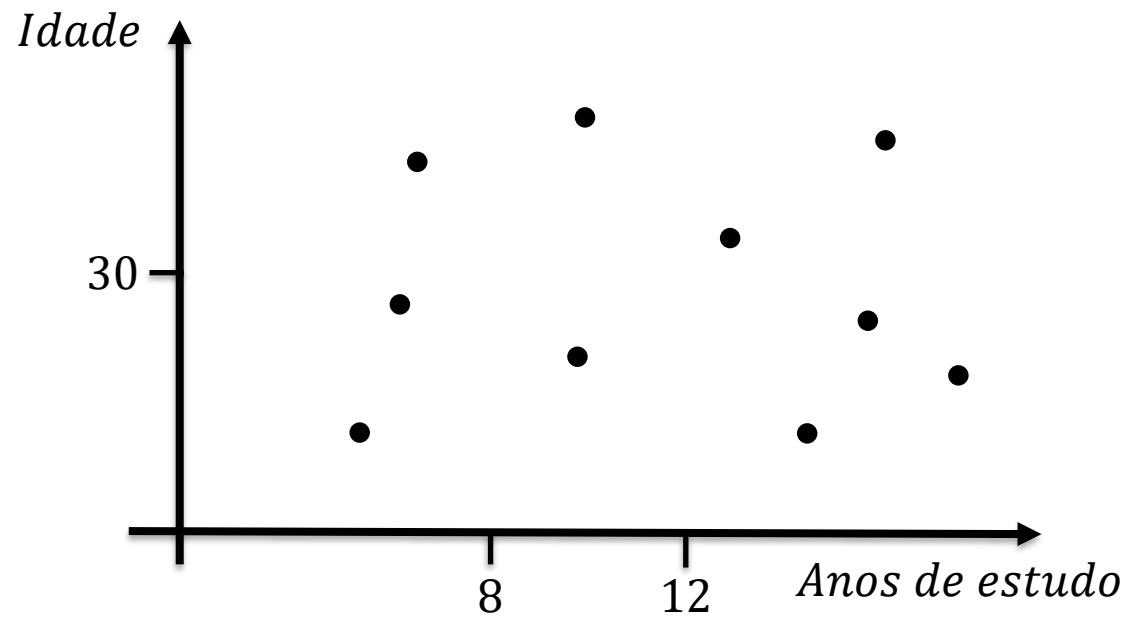


Regressão com Árvores de Decisão

- **Problemas de Regressão**

- Porém, a mesma técnica pode ser aplicada para problemas de regressão.

- **Exemplo:** estimar o salário de acordo com a idade e os anos de estudo.

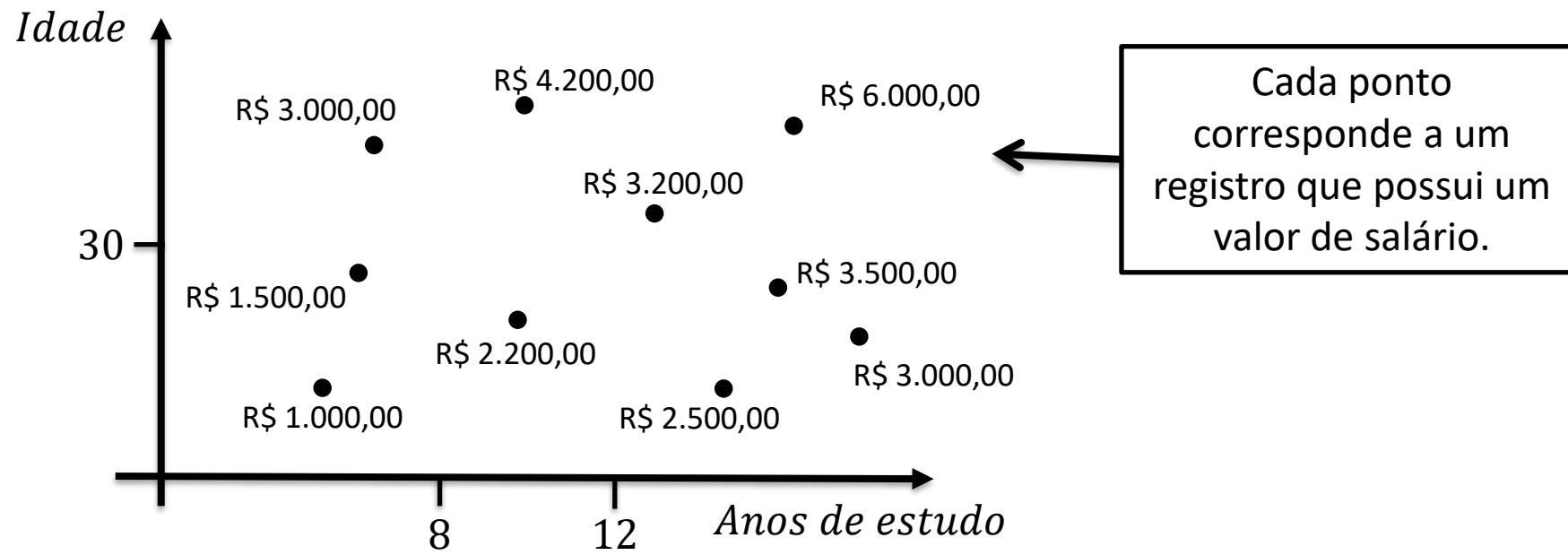


Regressão com Árvores de Decisão

- **Problemas de Regressão**

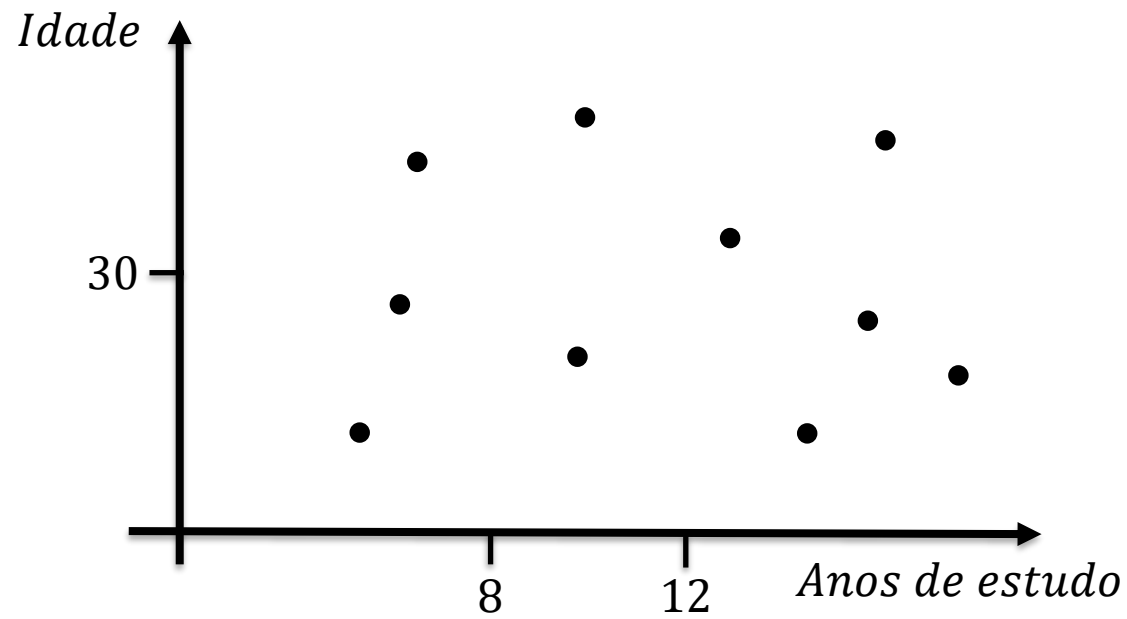
- Porém, a mesma técnica pode ser aplicada para problemas de regressão.

- **Exemplo:** estimar o salário de acordo com a idade e os anos de estudo.



Regressão com Árvores de Decisão

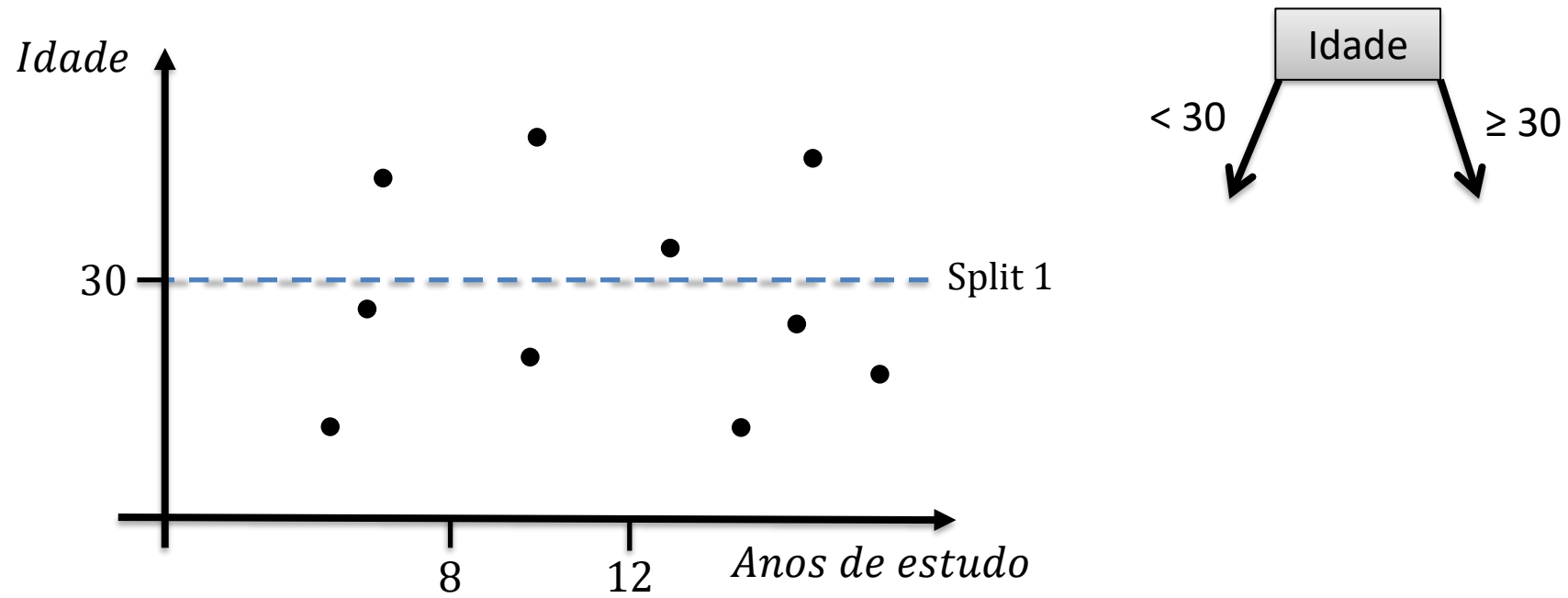
- **Problemas de Regressão**
 - A mesma estratégia utilizada para gerar as árvores de decisão é aplicada neste caso.
 - Usando os conceitos de ganho de entropia, os cortes são feitos:



Regressão com Árvores de Decisão

- **Problemas de Regressão**

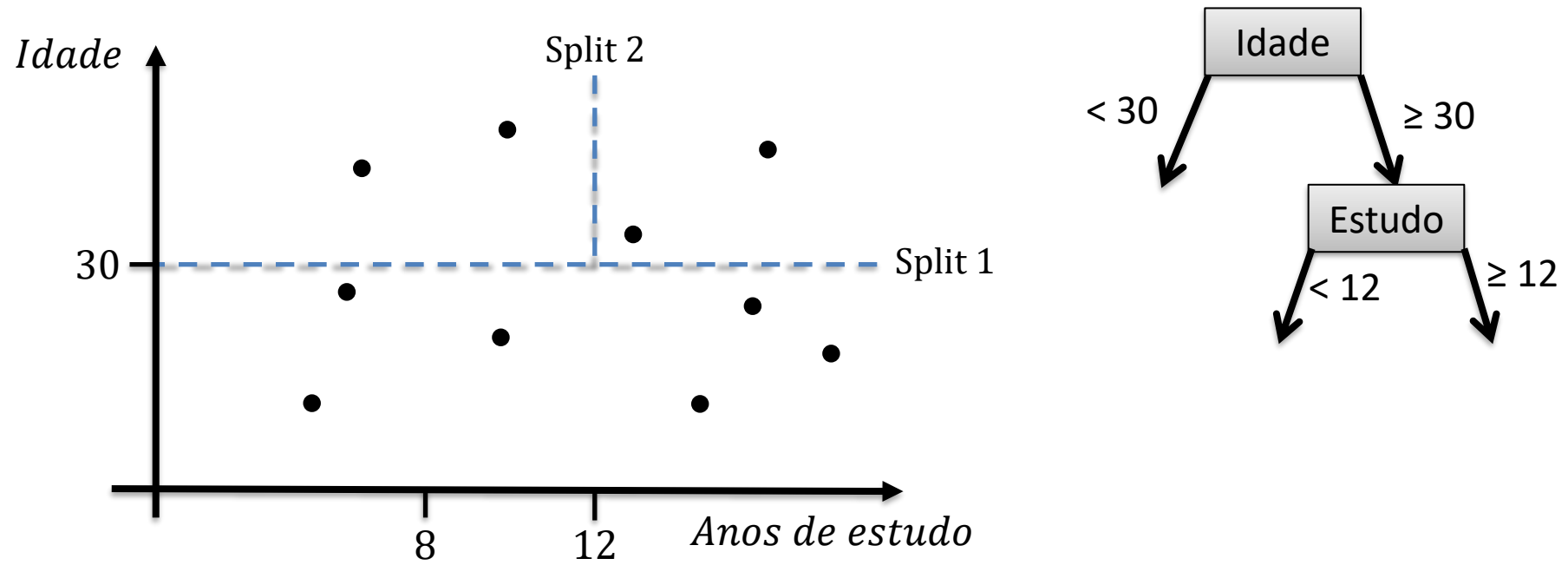
- A mesma estratégia utilizada para gerar as árvores de decisão é aplicada neste caso.
- Usando os conceitos de ganho de entropia, os cortes são feitos:



Regressão com Árvores de Decisão

- **Problemas de Regressão**

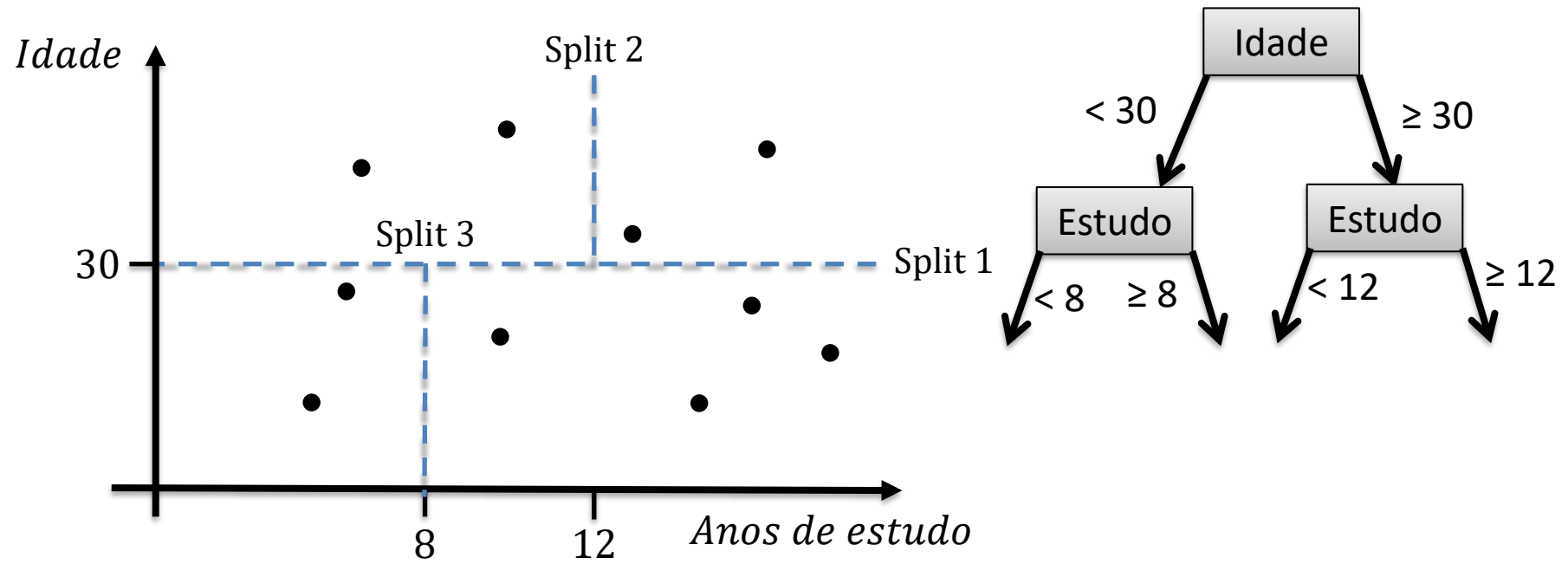
- A mesma estratégia utilizada para gerar as árvores de decisão é aplicada neste caso.
- Usando os conceitos de ganho de entropia, os cortes são feitos:



Regressão com Árvores de Decisão

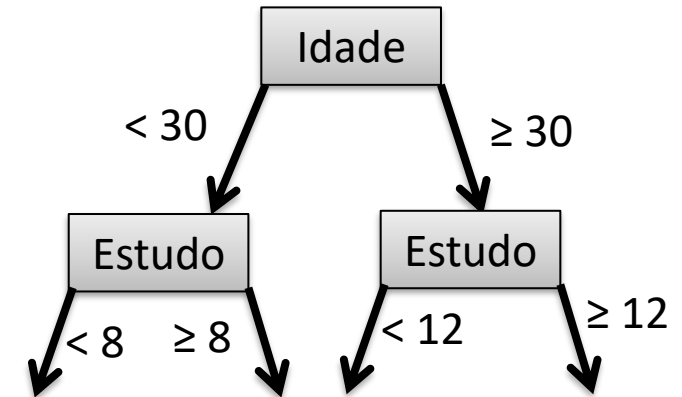
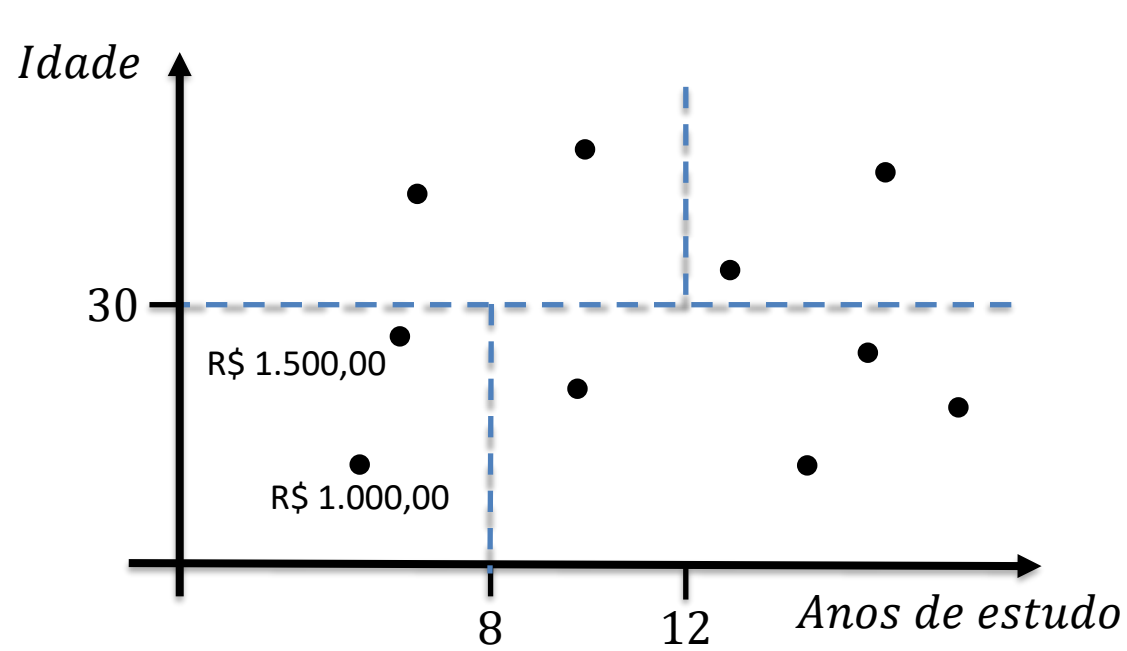
- **Problemas de Regressão**

- A mesma estratégia utilizada para gerar as árvores de decisão é aplicada neste caso.
- Usando os conceitos de ganho de entropia, os cortes são feitos:



Regressão com Árvores de Decisão

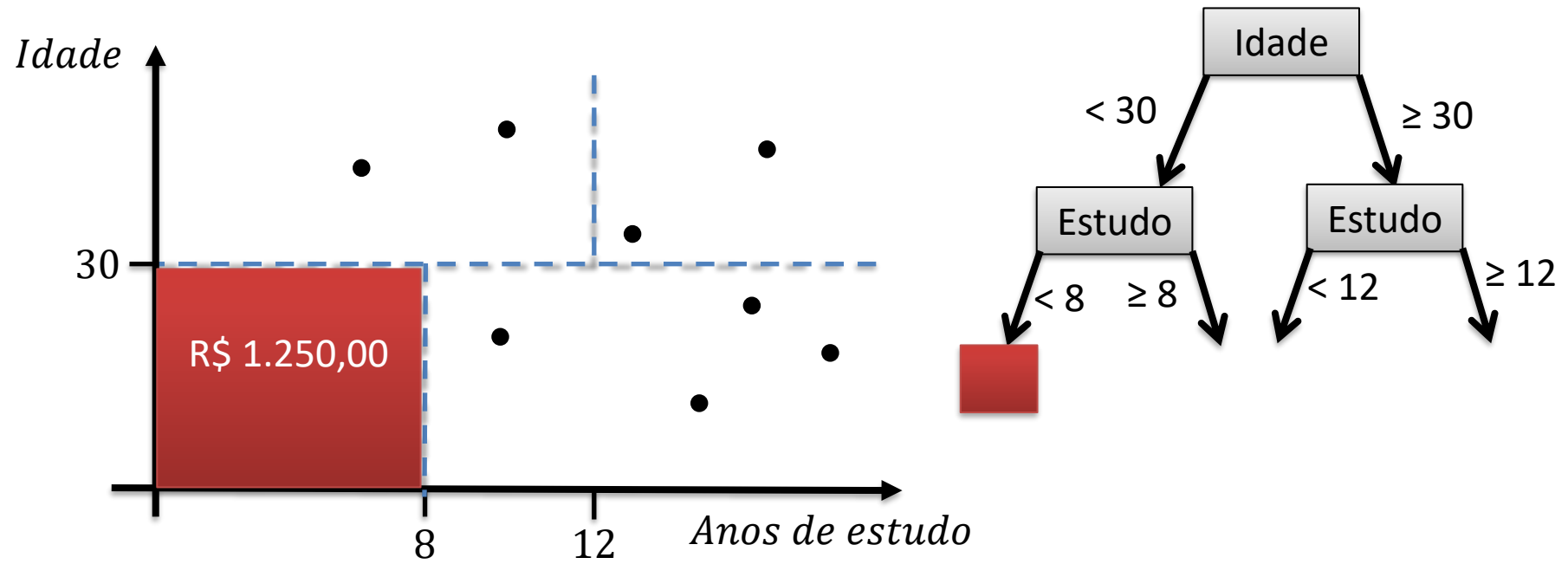
- **Problemas de Regressão**
 - Ao invés de representar uma classe, cada região irá corresponder à média dos valores dos registros que se encontram nela.



Regressão com Árvores de Decisão

- **Problemas de Regressão**

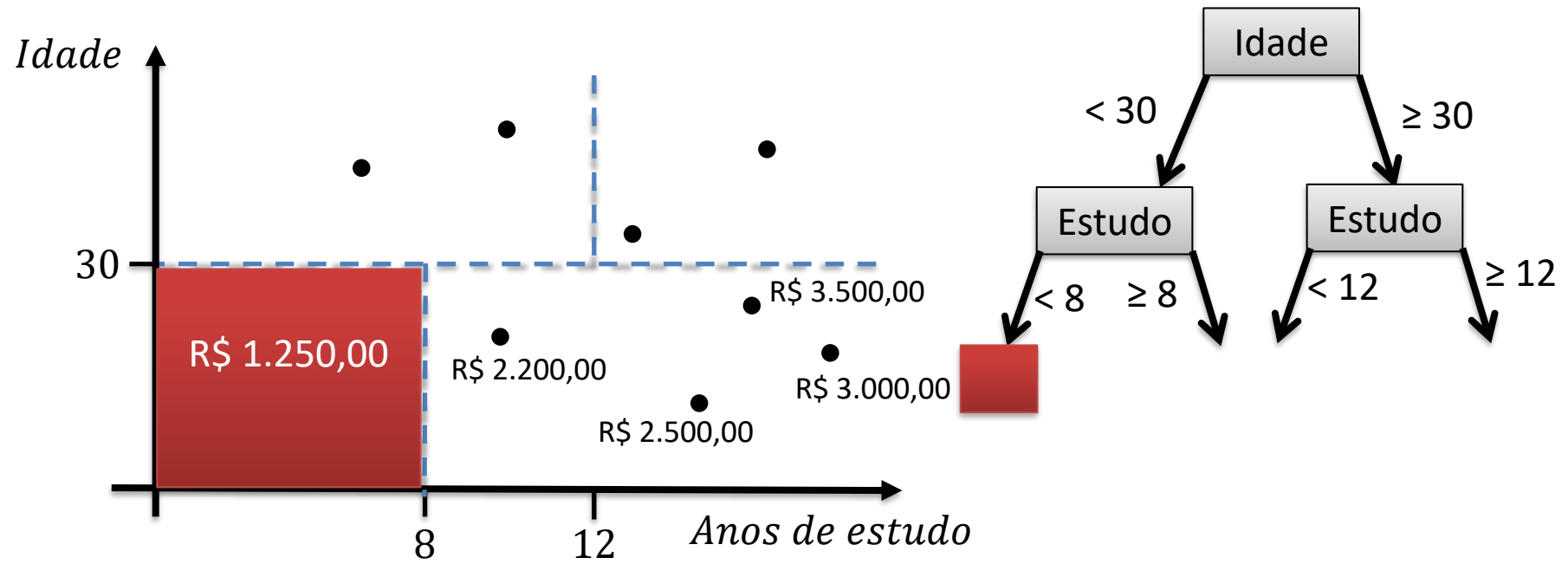
- Ao invés de representar uma classe, cada região irá corresponder à média dos valores dos registros que se encontram nela.



Regressão com Árvores de Decisão

- **Problemas de Regressão**

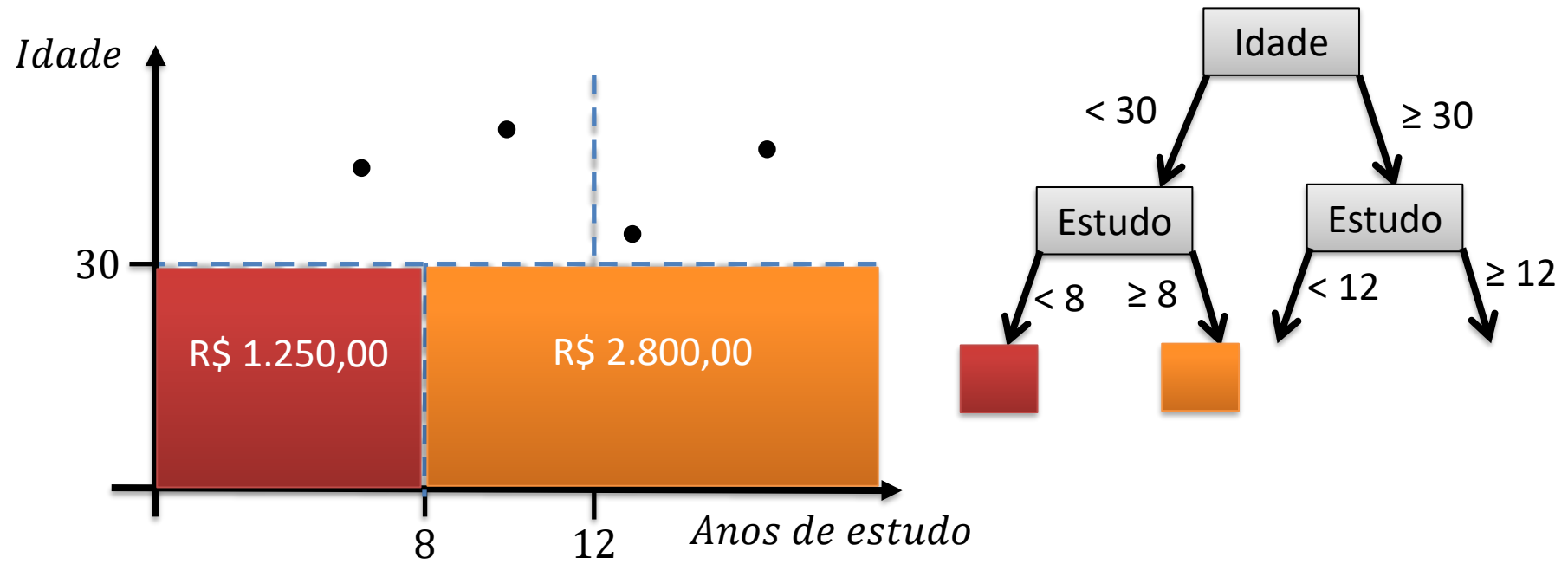
- Ao invés de representar uma classe, cada região irá corresponder à média dos valores dos registros que se encontram nela.



Regressão com Árvores de Decisão

- **Problemas de Regressão**

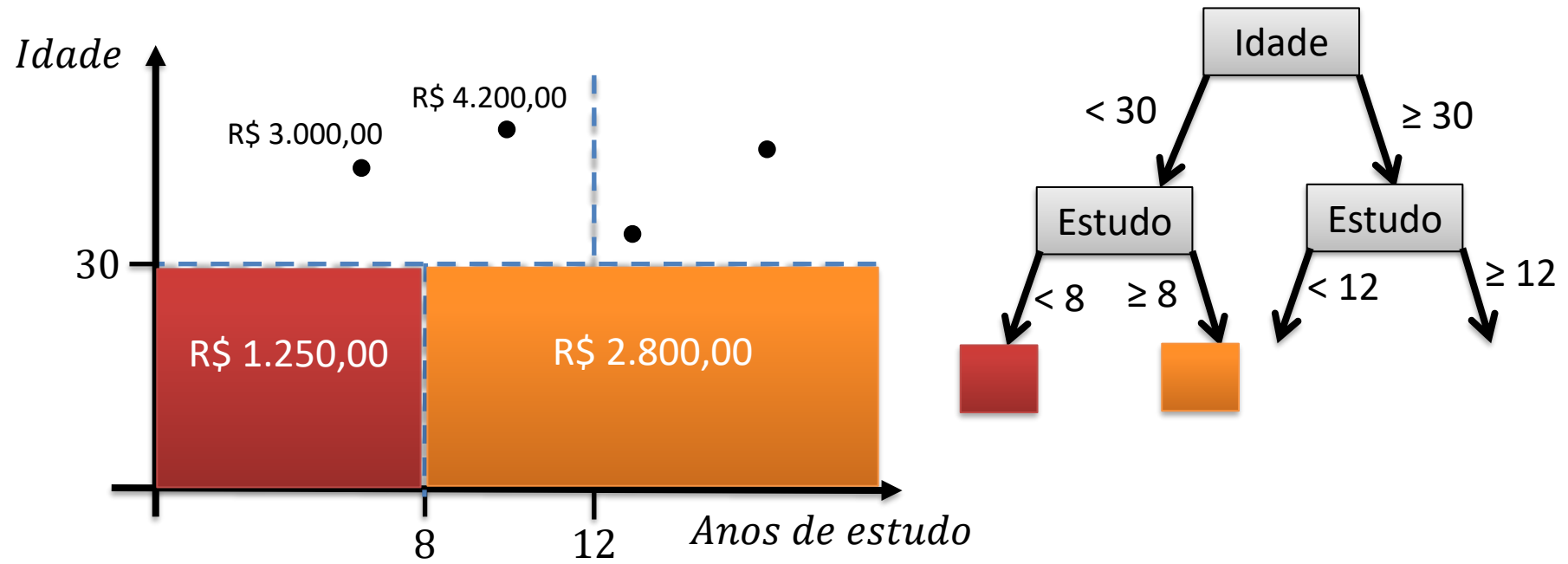
- Ao invés de representar uma classe, cada região irá corresponder à média dos valores dos registros que se encontram nela.



Regressão com Árvores de Decisão

- **Problemas de Regressão**

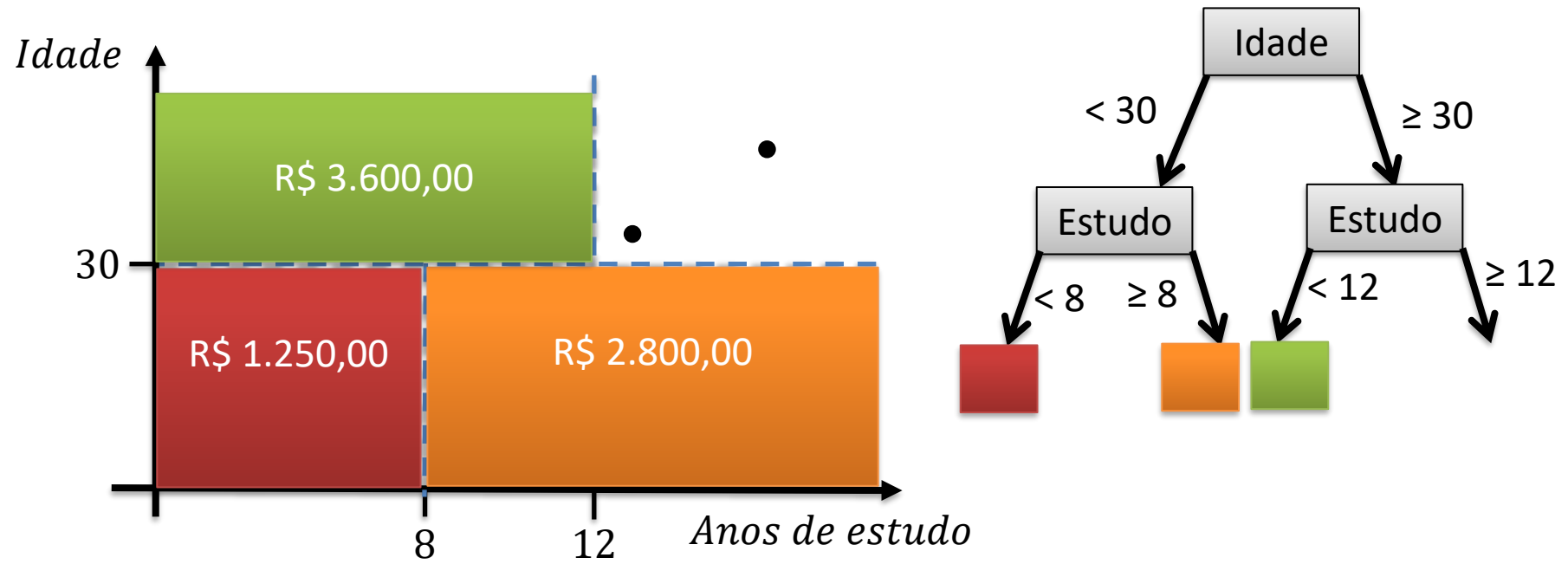
- Ao invés de representar uma classe, cada região irá corresponder à média dos valores dos registros que se encontram nela.



Regressão com Árvores de Decisão

- **Problemas de Regressão**

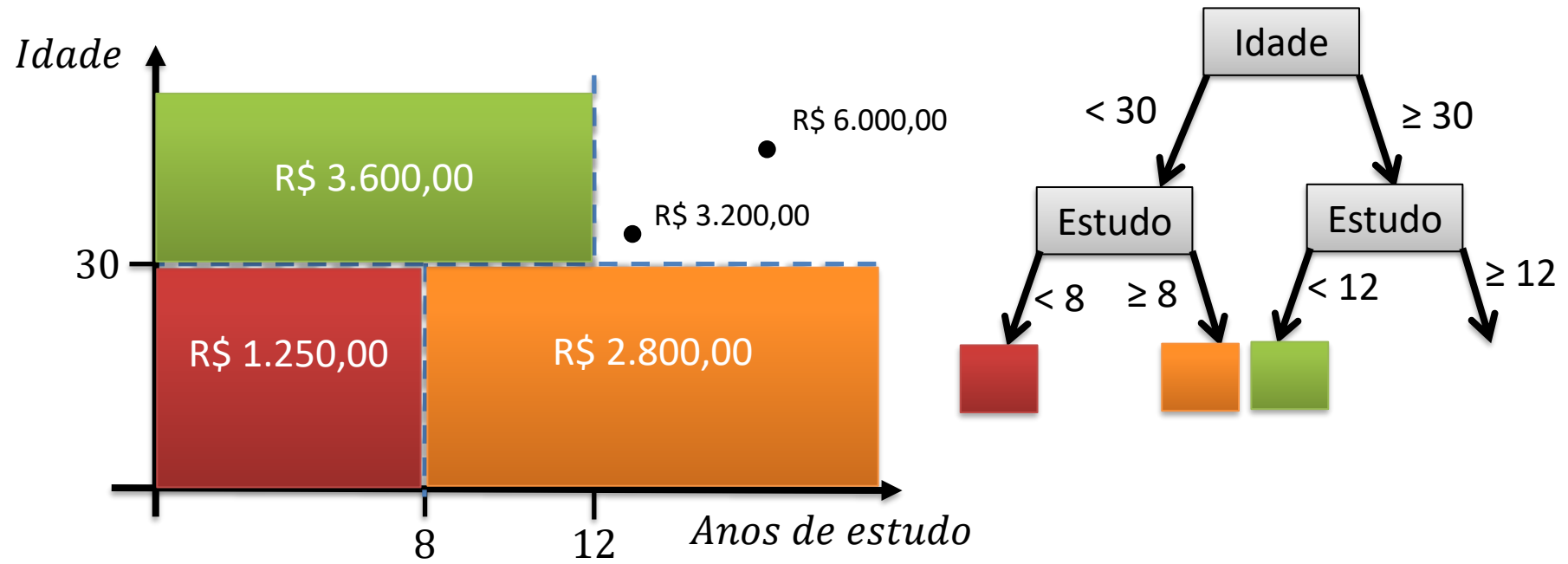
- Ao invés de representar uma classe, cada região irá corresponder à média dos valores dos registros que se encontram nela.



Regressão com Árvores de Decisão

- **Problemas de Regressão**

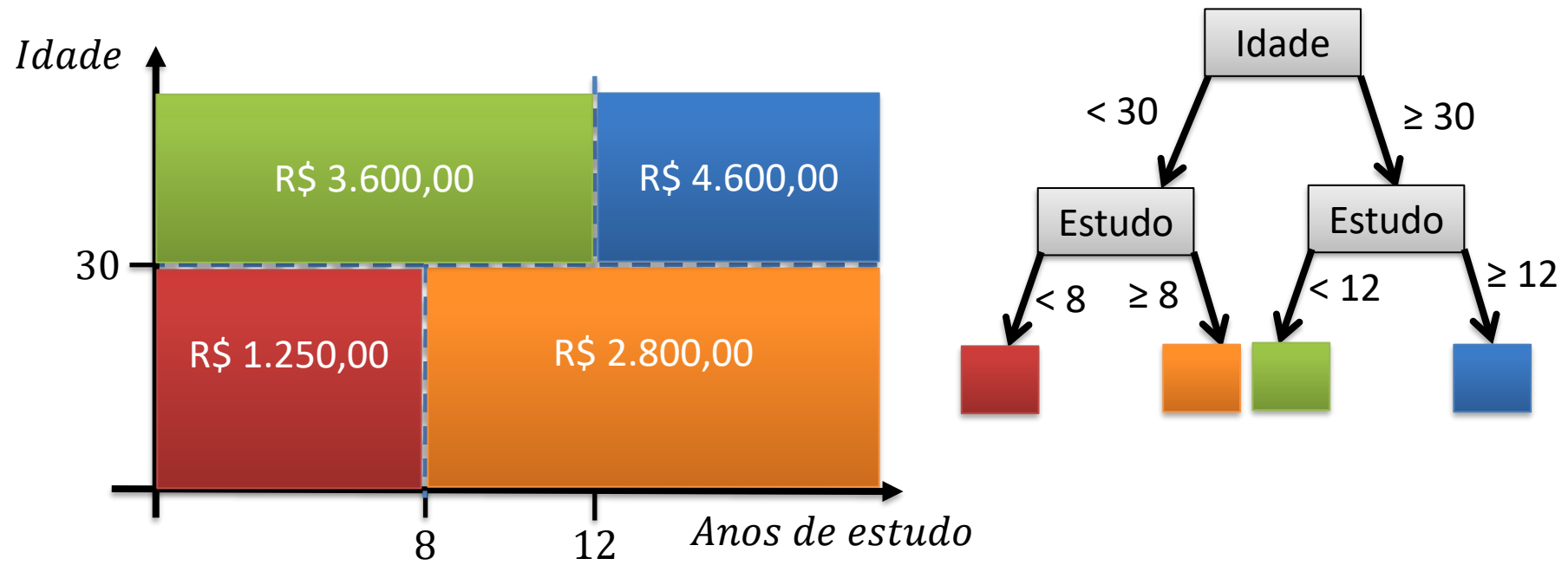
- Ao invés de representar uma classe, cada região irá corresponder à média dos valores dos registros que se encontram nela.



Regressão com Árvores de Decisão

- **Problemas de Regressão**

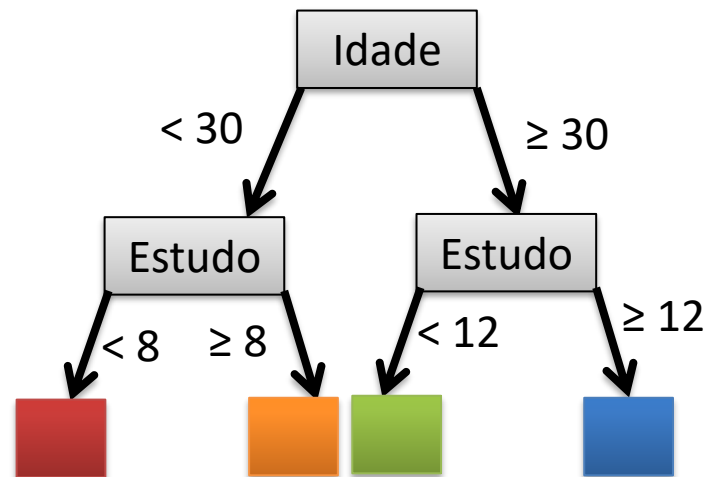
- Ao invés de representar uma classe, cada região irá corresponder à média dos valores dos registros que se encontram nela.



Regressão com Árvores de Decisão

- Problemas de Regressão

- O modelo criado pelas árvores de decisão **não é linear** nem **contínuo**, o que pode causar problemas em alguns casos específicos.
- Porém, assim como nos problemas de classificação, o modelo permite a avaliação das variáveis independentes que são **mais importantes** para o problema.



- **Regressão com Árvores de Decisão**

- Vamos aplicar a técnica para a base com valores referentes ao preço de um plano de saúde de acordo com a idade do cliente:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics
import numpy as np

##### Preprocessamento #####

base = pd.read_csv('plano_saude.csv')

# Separando dados em previsores e classes
cols_previsores = ['idade']

cols_objetivo = ['custo']
previsores = base[cols_previsores]
objetivo = base[cols_objetivo]

# Separando em base de testes e treinamento (usando 25% para teste)
from sklearn.model_selection import train_test_split
previsores_treinamento, previsores_teste, objetivo_treinamento, objetivo_teste = train_test_split(previsores,
                                                                                                    objetivo,
                                                                                                    test_size=0.25,
                                                                                                    random_state=0)

# Visualização dos dados
plt.scatter(previsores, objetivo)
plt.title('Regressão linear (dados completos)')
plt.xlabel('idade')
plt.ylabel('custo')
```

- **Regressão com Árvores de Decisão**

- Vamos aplicar a t
um plano de saúde

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics
import numpy as np
```

```
##### Preprocessamento
```

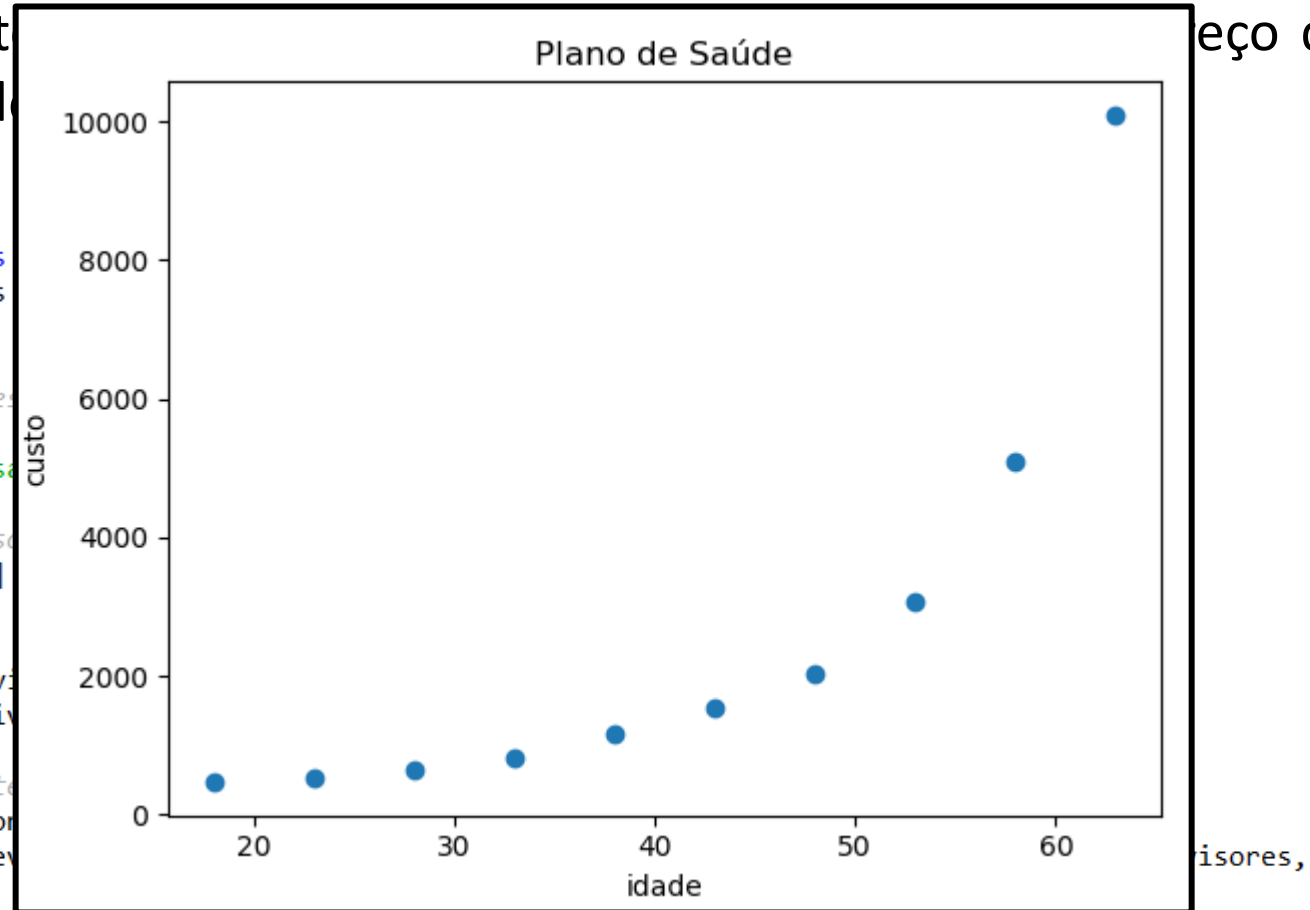
```
base = pd.read_csv('plano_saude.csv')
```

```
# Separando dados em previsores e objetivo
cols_previsores = ['idade']
```

```
cols_objetivo = ['custo']
previsores = base[cols_previsores]
objetivo = base[cols_objetivo]
```

```
# Separando em base de treinamento e teste
from sklearn.model_selection import train_test_split
previsores_treinamento, previsores_teste, objetivo_treinamento, objetivo_teste = train_test_split(previsores, objetivo, test_size=0.25, random_state=0)
```

```
# Visualização dos dados
plt.scatter(previsores, objetivo)
plt.title('Regressão linear (dados completos)')
plt.xlabel('idade')
plt.ylabel('custo')
```



test_size=0.25,
random_state=0)

- **Regressão com Árvores de Decisão**

- Para aplicar a Regressão com Árvores de Decisão, devemos fazer:

```
##### Regressão com Árvores de Decisão #####
```

```
from sklearn.tree import DecisionTreeRegressor  
regressor = DecisionTreeRegressor(max_depth=4,  
                                  random_state=0)
```

```
# Treinamento  
regressor.fit(previsores_treinamento, objetivo_treinamento)
```

```
# Teste  
previsoes = regressor.predict(previsores_teste)
```

- **Regressão com Árvores de Decisão**

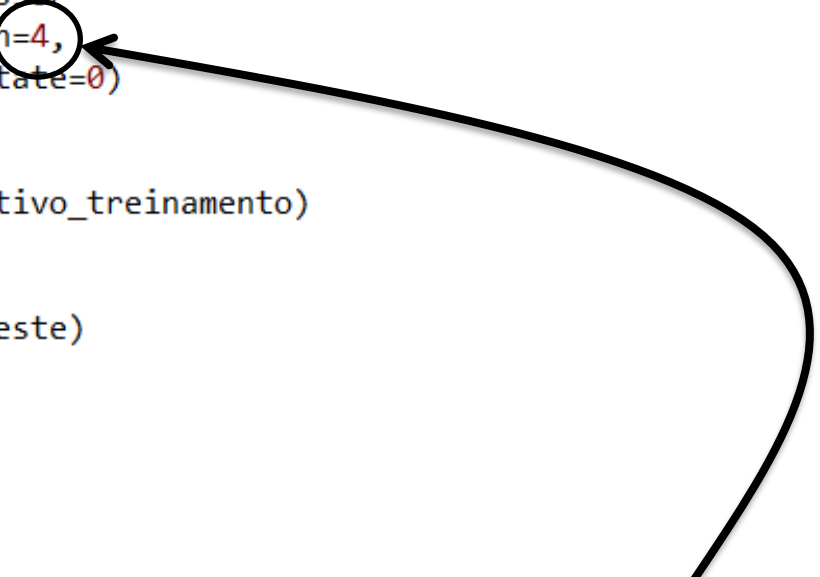
- Para aplicar a Regressão com Árvores de Decisão, devemos fazer:

```
##### Regressão com Árvores de Decisão #####
```

```
from sklearn.tree import DecisionTreeRegressor  
regressor = DecisionTreeRegressor(max_depth=4,  
                                  random_state=0)
```

```
# Treinamento  
regressor.fit(previsores_treinamento, objetivo_treinamento)
```

```
# Teste  
previsoes = regressor.predict(previsores_teste)
```



Parâmetro de configuração que
corresponde à altura máxima da árvore
criada.

- **Regressão com Árvores de Decisão**

- Para analisar os resultados, podemos plotar o gráfico das previsões:

```
##### Avaliação dos resultados #####

# Visualização dos dados
import numpy as np
X_plot = np.arange(previsores.min().values[0], previsores.max().values[0], 0.1)
X_plot = X_plot.reshape(-1, 1)
Y_plot = regressor.predict(X_plot)
plt.scatter(previsores_treinamento, objetivo_treinamento)
plt.plot(X_plot, Y_plot, color = 'red')
plt.title('Regressão com árvores')
plt.xlabel('Idade')
plt.ylabel('Custo')

score = regressor.score(previsores_teste, objetivo_teste)
mae = metrics.mean_absolute_error(objetivo_teste, previsoes)
mse = metrics.mean_squared_error(objetivo_teste, previsoes)
rmse = np.sqrt(metrics.mean_squared_error(objetivo_teste, previsoes))

print('Score:', score)
print('Mean Absolute Error:', mae)
print('Mean Squared Error:', mse)
print('Root Mean Squared Error:', rmse)
```

- **Regressão com Árvores de Decisão**

- Para analisar os resultados, podemos plotar o gráfico das previsões:

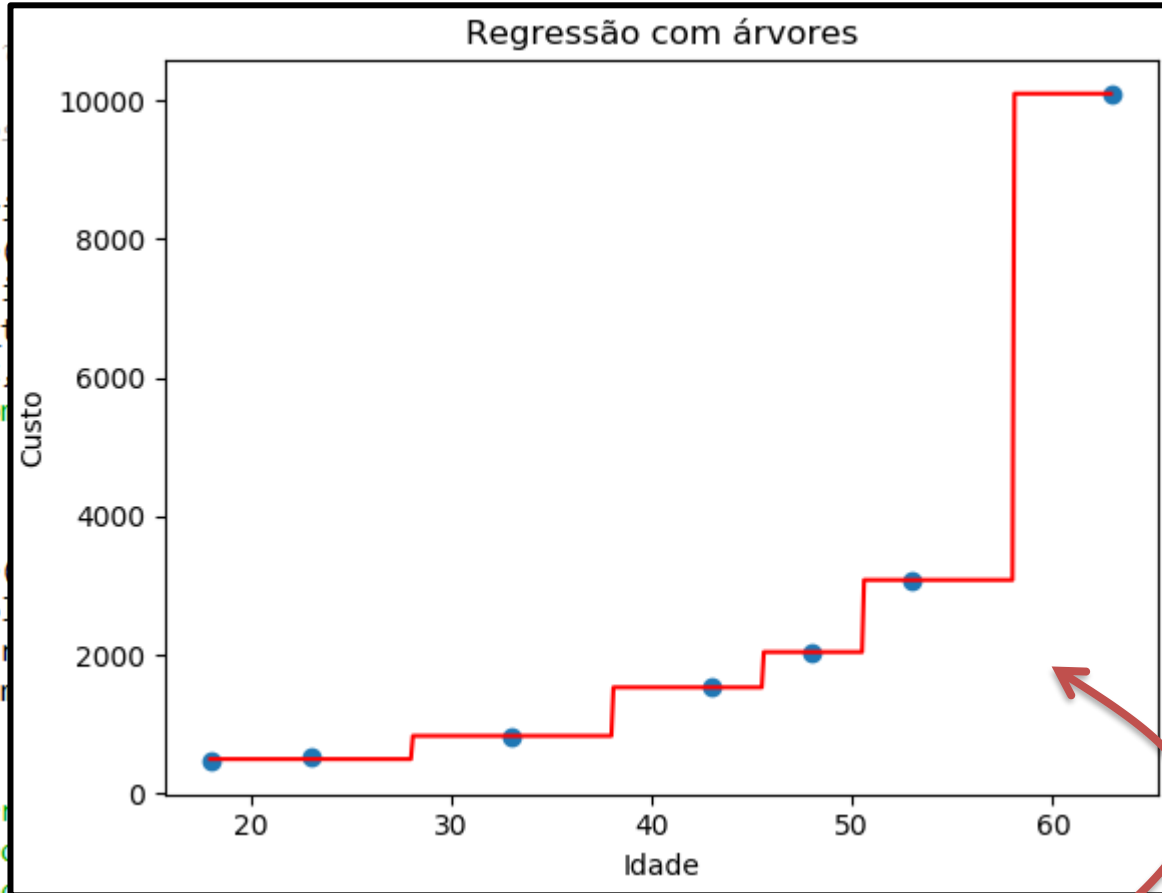
```
##### Avaliação
```

```
# Visualização dos dados
```

```
import numpy as np
X_plot = np.arange(previsores_t.shape[0])
X_plot = X_plot.reshape((previsores_t.shape[0], 1))
Y_plot = regressor.predict(X_plot)
plt.scatter(previsores_t, Y_plot)
plt.plot(X_plot, Y_plot, color='red', linestyle='step')
plt.title('Regressão com Árvores de Decisão')
plt.xlabel('Idade')
plt.ylabel('Custo')
```

```
score = regressor.score(previsores_t, Y_plot)
mae = metrics.mean_absolute_error(Y_plot, Y_plot)
mse = metrics.mean_squared_error(Y_plot, Y_plot)
rmse = np.sqrt(mse)
```

```
print('Score:', score)
print('Mean Absolute Error:', mae)
print('Mean Squared Error:', mse)
print('Root Mean Squared Error:', rmse)
```



```
Score: 0.6484870501871489
Mean Absolute Error: 825.0
Mean Squared Error: 1400341.6666666667
Root Mean Squared Error: 1183.3603283305836
```

Neste caso simples, é possível visualizar o modelo criado pela Árvore de Decisão.

- **Regressão com Árvores de Decisão**

– Voltando ao problema dos preços de imóveis, o pré-processamento fica igual.

```
import pandas as pd

##### #Pré-processamento dos dados #####

#Leitura dos dados
base = pd.read_csv('house_prices.csv')
base.describe()

# =====
#                               Tratando valores inválidos
# =====
# Não há valores inconsistentes

# Procurando as colunas que possuem algum valor faltante
pd.isnull(base).any()

# =====
#                               Separando dados em previsores e objetivo
# =====

cols_previsores = ['bedrooms', 'bathrooms', 'sqft_living', 'sqft_lot',
                   'floors', 'waterfront', 'view', 'condition', 'grade', 'sqft_above',
                   'sqft_basement', 'yr_built', 'yr_renovated', 'zipcode', 'lat', 'long']
# Não usei: date, sqft_living15 e sqft_lot15

cols_objetivo = ['price']
previsores = base[cols_previsores]
objetivo = base[cols_objetivo]

# =====
#                               Transformar as variáveis categóricas em valores numéricos
# =====
# Todas as variáveis são numéricas

# =====
#                               Separando em base de testes e treinamento
# =====

# usando 25% para teste
from sklearn.model_selection import train_test_split
previsores_treinamento, previsores_teste, objetivo_treinamento, objetivo_teste = train_test_split(previsores,
                                                                                                  objetivo,
                                                                                                  test_size=0.25,
                                                                                                  random_state=0)

# =====
#                               Padronização dos dados
# =====

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
previsores_treinamento = scaler.fit_transform(previsores_treinamento)
previsores_teste = scaler.transform(previsores_teste)
```


- **Regressão com Árvores de Decisão**

- Basta repetir o procedimento de regressão, utilizando os melhores valores encontrados para os parâmetros de configuração.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics
import numpy as np

##### Preprocessamento #####

# Arquivo separado

##### Regressão com Árvores de Decisão #####

from sklearn.tree import DecisionTreeRegressor
regressor = DecisionTreeRegressor(max_depth=?
                                  random_state=0)

# Treinamento
regressor.fit(previsores_treinamento, objetivo_treinamento)

# Teste
previsoes = regressor.predict(previsores_teste)

##### Avaliação dos resultados #####

score = regressor.score(previsores_teste, objetivo_teste)
mae = metrics.mean_absolute_error(objetivo_teste, previsoes)
mse = metrics.mean_squared_error(objetivo_teste, previsoes)
rmse = np.sqrt(metrics.mean_squared_error(objetivo_teste, previsoes))

print('Score:', score)
print('Mean Absolute Error:', mae)
print('Mean Squared Error:', mse)
print('Root Mean Squared Error:', rmse)
```

- **Regressão com Árvores de Decisão**

- Basta repetir o procedimento de regressão, utilizando os melhores valores encontrados para os parâmetros de configuração.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics
import numpy as np

##### Preprocessamento #####

# Arquivo separado

##### Regressão com Árvores de Decisão #####

from sklearn.tree import DecisionTreeRegressor
regressor = DecisionTreeRegressor(max_depth=?
                                  random_state=0)

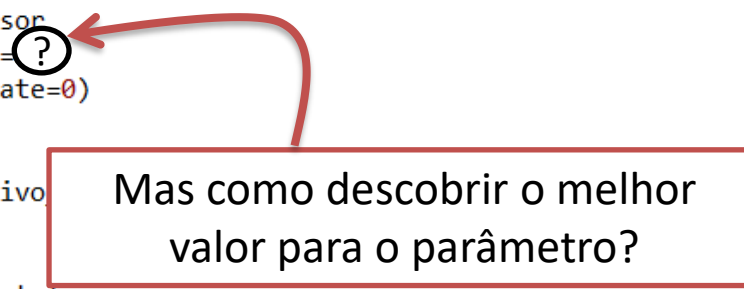
# Treinamento
regressor.fit(previsores_treinamento, objetivo_treinamento)

# Teste
previsoes = regressor.predict(previsores_teste)

##### Avaliação dos resultados #####

score = regressor.score(previsores_teste, objetivo_teste)
mae = metrics.mean_absolute_error(objetivo_teste, previsoes)
mse = metrics.mean_squared_error(objetivo_teste, previsoes)
rmse = np.sqrt(metrics.mean_squared_error(objetivo_teste, previsoes))

print('Score:', score)
print('Mean Absolute Error:', mae)
print('Mean Squared Error:', mse)
print('Root Mean Squared Error:', rmse)
```



Mas como descobrir o melhor valor para o parâmetro?

- **Regressão com Árvores de Decisão**

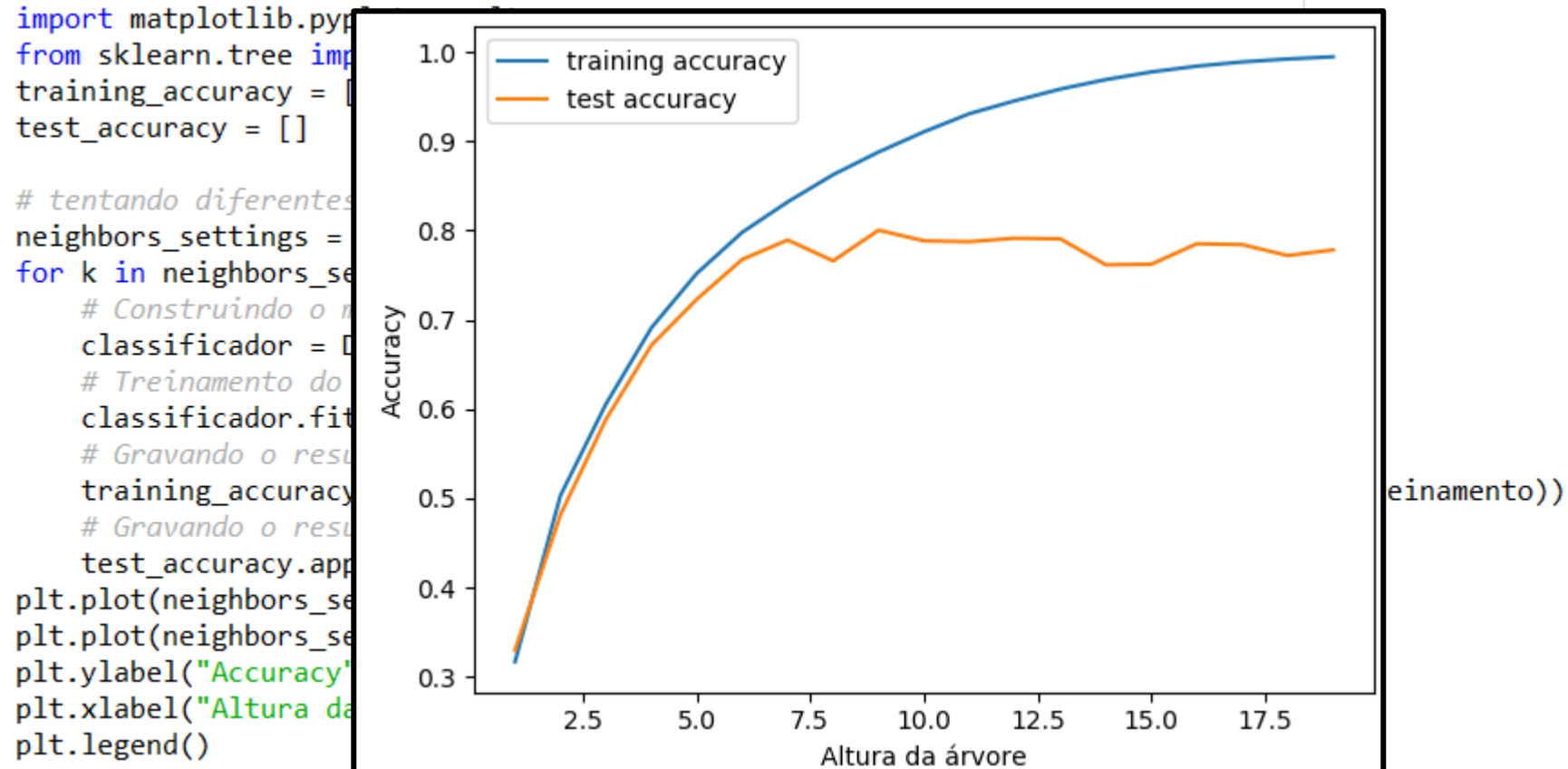
- Podemos fazer uma investigação para encontrar um bom parâmetro, que não cause overfitting ou underfitting:

```
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeRegressor
training_accuracy = []
test_accuracy = []

# tentando diferentes valores de K: de 1 to 20
neighbors_settings = range(1, 20)
for k in neighbors_settings:
    # Construindo o modelo
    classificador = DecisionTreeRegressor(max_depth=k, random_state=0)
    # Treinamento do modelo
    classificador.fit(previsores_treinamento, objetivo_treinamento)
    # Gravando o resultado para os dados de treinamento
    training_accuracy.append(classificador.score(previsores_treinamento, objetivo_treinamento))
    # Gravando o resultado para os dados de teste (generalização)
    test_accuracy.append(classificador.score(previsores_teste, objetivo_teste))
plt.plot(neighbors_settings, training_accuracy, label="training accuracy")
plt.plot(neighbors_settings, test_accuracy, label="test accuracy")
plt.ylabel("Accuracy")
plt.xlabel("Altura da árvore")
plt.legend()
```

- **Regressão com Árvores de Decisão**

- Podemos fazer uma investigação para encontrar um bom parâmetro, que não cause overfitting ou underfitting:



- **Regressão com Árvores de Decisão**

- Podemos fazer uma investigação para encontrar um bom parâmetro, que não cause overfitting ou underfitting:

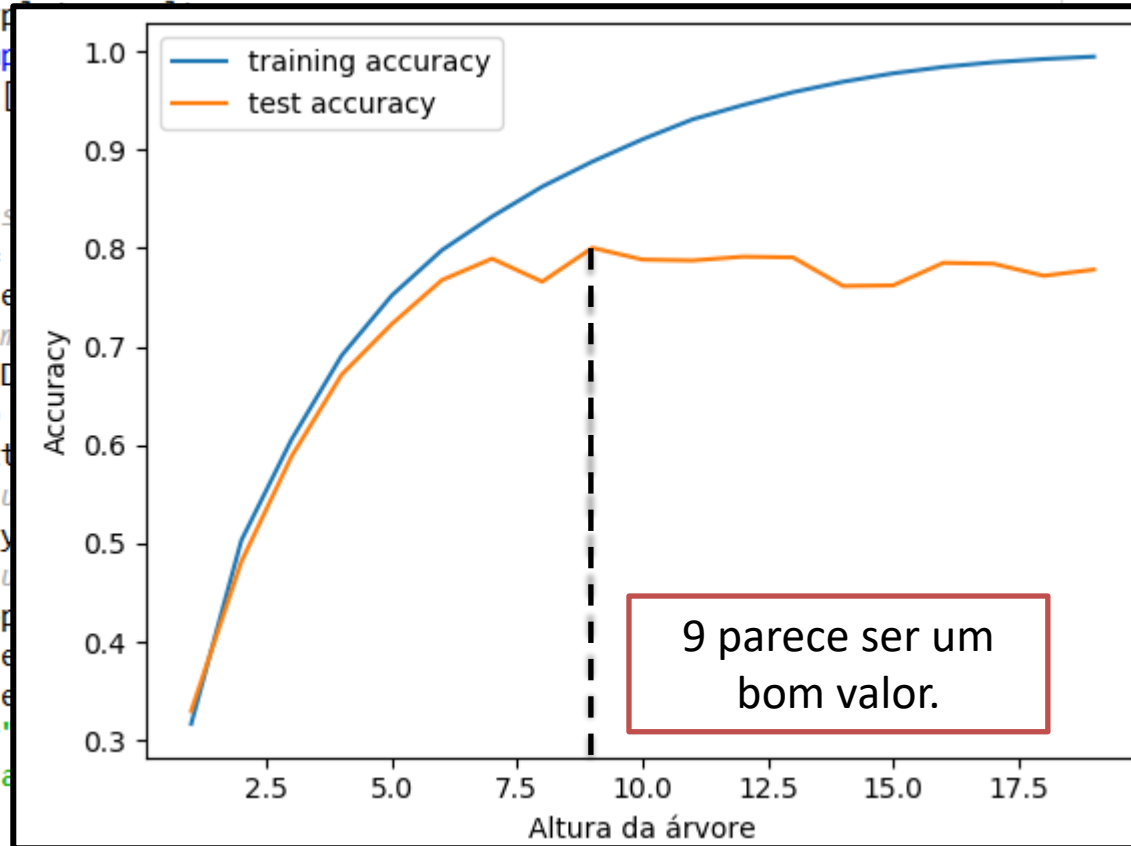
```
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier

training_accuracy = []
test_accuracy = []

# tentando diferentes valores de k
neighbors_settings = range(1, 21)

for k in neighbors_settings:
    # Construindo o modelo
    classificador = DecisionTreeClassifier(max_depth=k)
    # Treinamento do modelo
    classificador.fit(X_train, y_train)
    # Gravando o resultado do treinamento
    training_accuracy.append(classificador.score(X_train, y_train))
    # Gravando o resultado da validação
    test_accuracy.append(classificador.score(X_test, y_test))

plt.plot(neighbors_settings, training_accuracy, label="training accuracy")
plt.plot(neighbors_settings, test_accuracy, label="test accuracy")
plt.ylabel("Accuracy")
plt.xlabel("Altura da árvore")
plt.legend()
```



einamento))

- **Regressão com Árvores de Decisão**

- Basta repetir o procedimento de regressão, utilizando os melhores valores encontrados para os parâmetros de configuração.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics
import numpy as np

##### Preprocessamento #####

# Arquivo separado

##### Regressão com Árvores de Decisão #####

from sklearn.tree import DecisionTreeRegressor
regressor = DecisionTreeRegressor(max_depth=9,
                                  random_state=0)

# Treinamento
regressor.fit(previsores_treinamento, objetivo_treinamento)

# Teste
previsoes = regressor.predict(previsores_teste)

##### Avaliação dos resultados #####

score = regressor.score(previsores_teste, objetivo_teste)
mae = metrics.mean_absolute_error(objetivo_teste, previsoes)
mse = metrics.mean_squared_error(objetivo_teste, previsoes)
rmse = np.sqrt(metrics.mean_squared_error(objetivo_teste, previsoes))

print('Score:', score)
print('Mean Absolute Error:', mae)
print('Mean Squared Error:', mse)
print('Root Mean Squared Error:', rmse)
```

- **Regressão**

- Basta repetir valores e

Vamos registrar os resultados em um arquivo único, para que possamos compará-lo com resultados futuros:

Score: 0.8002473584948447
Mean Absolute Error: 91260.69407473727
Mean Squared Error: 26535258326.21756
Root Mean Squared Error: 162896.46505132504

```
import numpy as np
```

```
##### Preprocessamento #####
```

House Sales in King County, USA				
Descrição		Dados e imóveis e seus respectivos valores em um distrito dos Estados Unidos.		
Total de registros:		21613		
Total de características:		16		
Total de colunas objetivo:		1		
Resultados				
Método	Configuração	Tratamento de dados	Score %	RMSE
Regressão Linear	-	Padronização	0.69	202633.35
Regressão Polinomial	Grau 2	Padronização	0.8083	159537.82
Árvores de Decisão	max_depth=9	Padronização	0.8002	162896.46

```
print('Mean Squared Error:', mse)  
print('Root Mean Squared Error:', rmse)
```

os melhores

- **Construindo o script**

- Além do resultado, você pode executar as seguintes linhas para salvar e exibir a árvore construída.

```
# Exportando a árvore para fazer figura
from sklearn.tree import export_graphviz
export_graphviz(regressor, out_file="tree.dot",
                feature_names=cols_previsores,
                impurity=False, filled=True)

# Visualizando a árvore
import graphviz
with open("tree.dot") as f:
    dot_graph = f.read()
display(graphviz.Source(dot_graph))
```

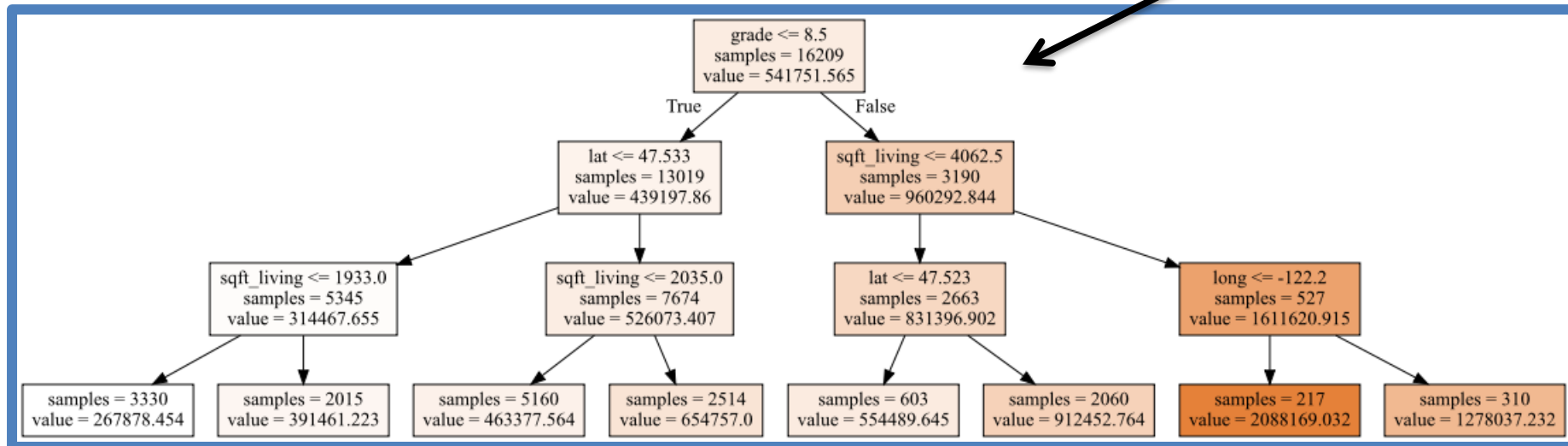

- **Construindo o script**

- Além do resultado, você pode executar as seguintes linhas para salvar e exibir a árvore construída.

```
# Exportando a árvore para fazer figura
from sklearn.tree import export_graphviz
export_graphviz(regressor, out_file="tree.dot",
                feature_names=cols_previsores,
                impurity=False, filled=True)

# Visualizando a árvore
import graphviz
with open("tree.dot") as f:
    dot_graph = f.read()
display(graphviz.Source(dot_graph))
```

Exemplo de
árvore usando
profundidade
máxima de 3.



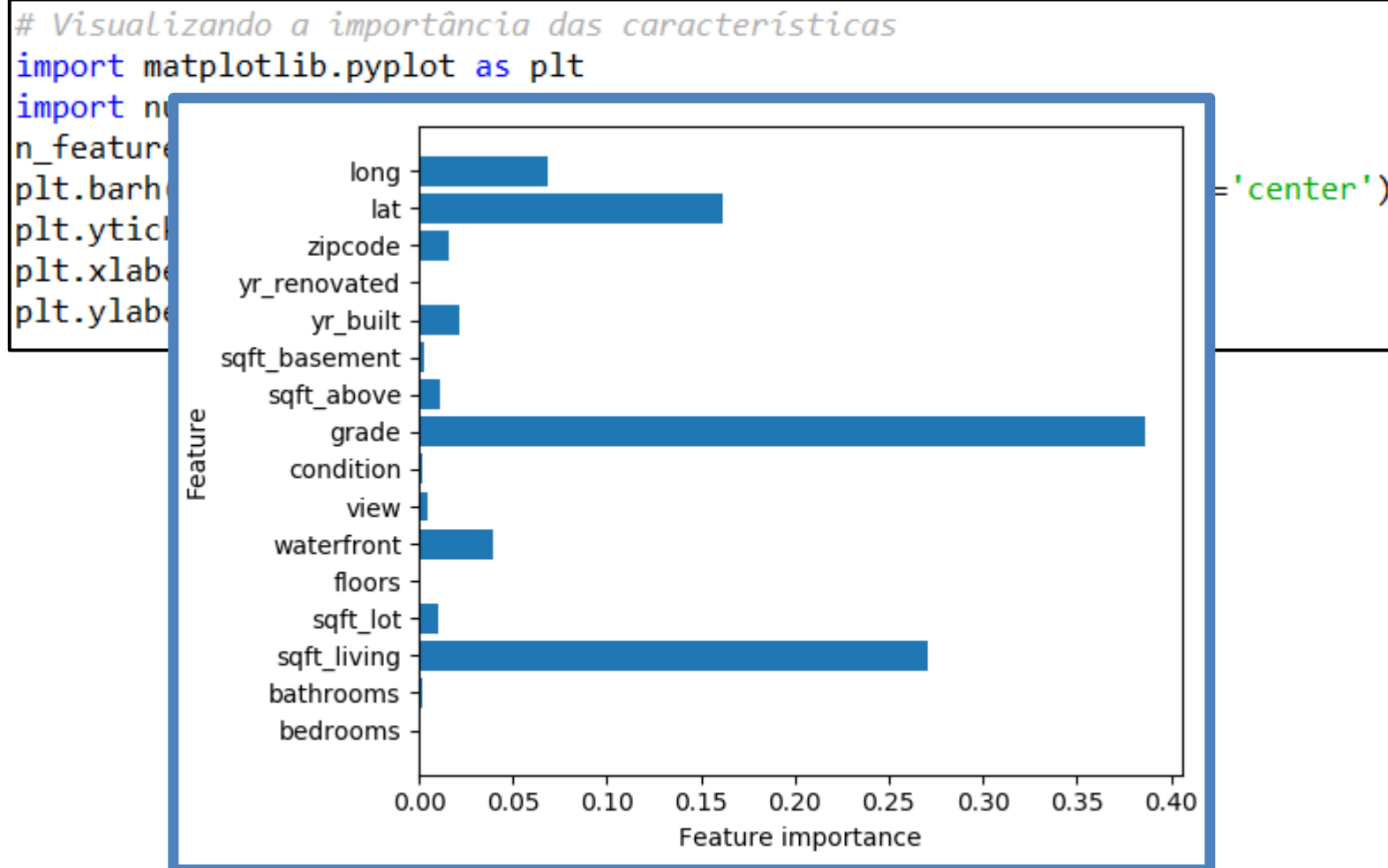
- **Construindo o script**

- Também é interessante gerar um gráfico com a importância que cada característica da base teve no resultado final.
- Para isso, acrescente as seguintes linhas ao final do script:

```
# Visualizando a importância das características  
import matplotlib.pyplot as plt  
import numpy as np  
n_features = previsores.columns.size  
plt.barh(range(n_features), regressor.feature_importances_, align='center')  
plt.yticks(np.arange(n_features), previsores.columns)  
plt.xlabel("Feature importance")  
plt.ylabel("Feature")
```

- **Construindo o script**

- Também é interessante gerar um gráfico com a importância que cada característica da base teve no resultado final.
- Para isso, acrescente as seguintes linhas ao final do script:



Regressão com Random Forest

Regressão com Random Forest

- **Ensemble learning**
 - Tipo de aprendizado que corresponde à analogia de consultar **diversos profissionais especialistas** para tomar uma decisão.
 - São aplicados os mesmos critérios que usamos para criar uma floresta de árvores de decisão para problemas de classificação.
 - Basicamente, consiste na utilização de vários algoritmos **juntos** para construir um mais poderoso.



Regressão com Random Forest

- **Descrição da técnica**

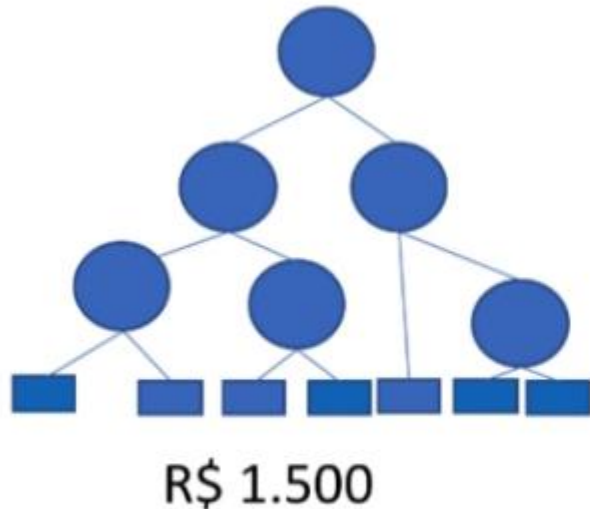
- As árvores de uma floresta randômica são geradas adicionando-se um pouco de **aleatoriedade** em sua construção.
- Normalmente, existem **duas maneiras** de inserir a aleatoriedade:
 - Na seleção aleatória de **partições dos registros** para a construção da árvore.
 - Na seleção aleatória de **características usadas** para cada split.
- O **número de árvores** na floresta é um dos **parâmetros** do algoritmo.

Regressão com Random Forest

- **Descrição da técnica**
 - Em problemas de regressão, a resposta final será a média da resposta de cada árvore da floresta.
 - Por exemplo: qual é a previsão de salário para a pessoa X?

Regressão com Random Forest

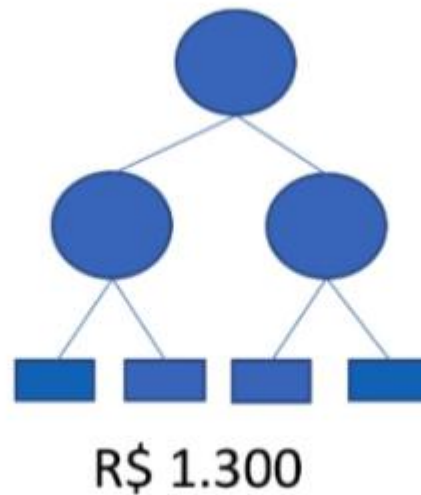
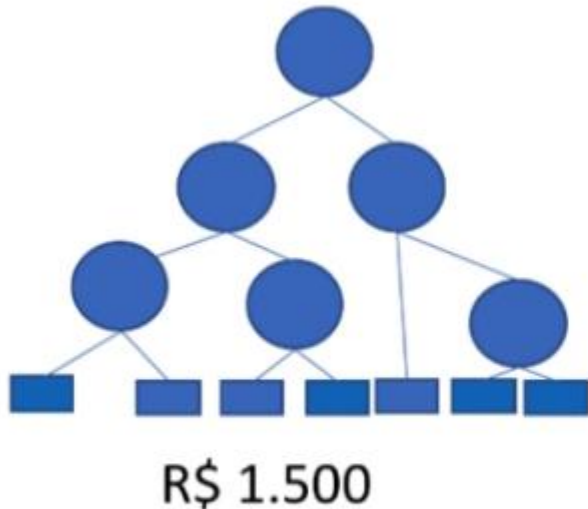
- **Descrição da técnica**
 - Em problemas de regressão, a resposta final será a média da resposta de cada árvore da floresta.
 - Por exemplo: qual é a previsão de salário para a pessoa X?



Regressão com Random Forest

- **Descrição da técnica**

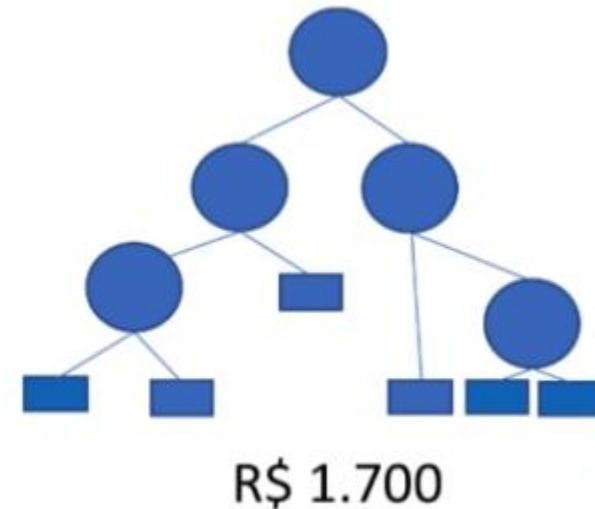
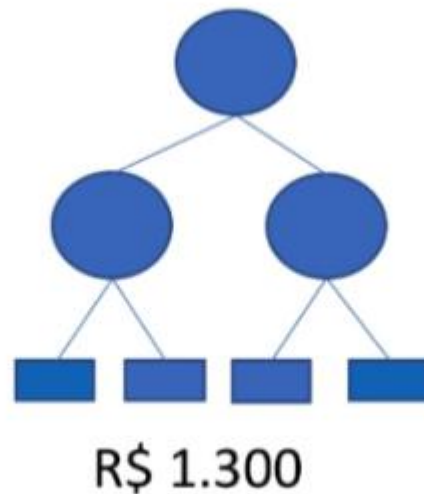
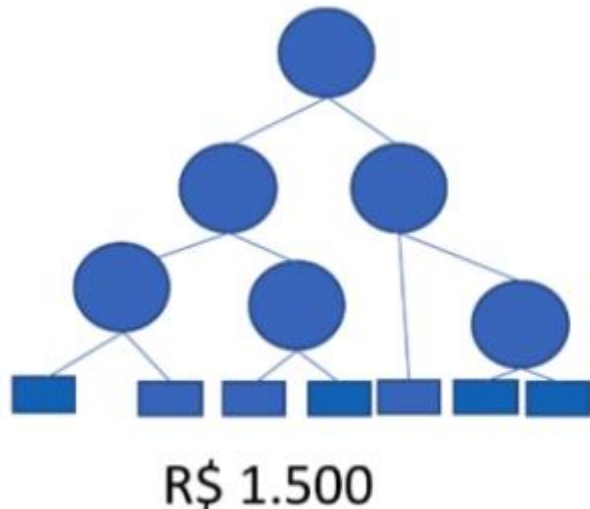
- Em problemas de regressão, a resposta final será a média da resposta de cada árvore da floresta.
- Por exemplo: qual é a previsão de salário para a pessoa X?



Regressão com Random Forest

- **Descrição da técnica**

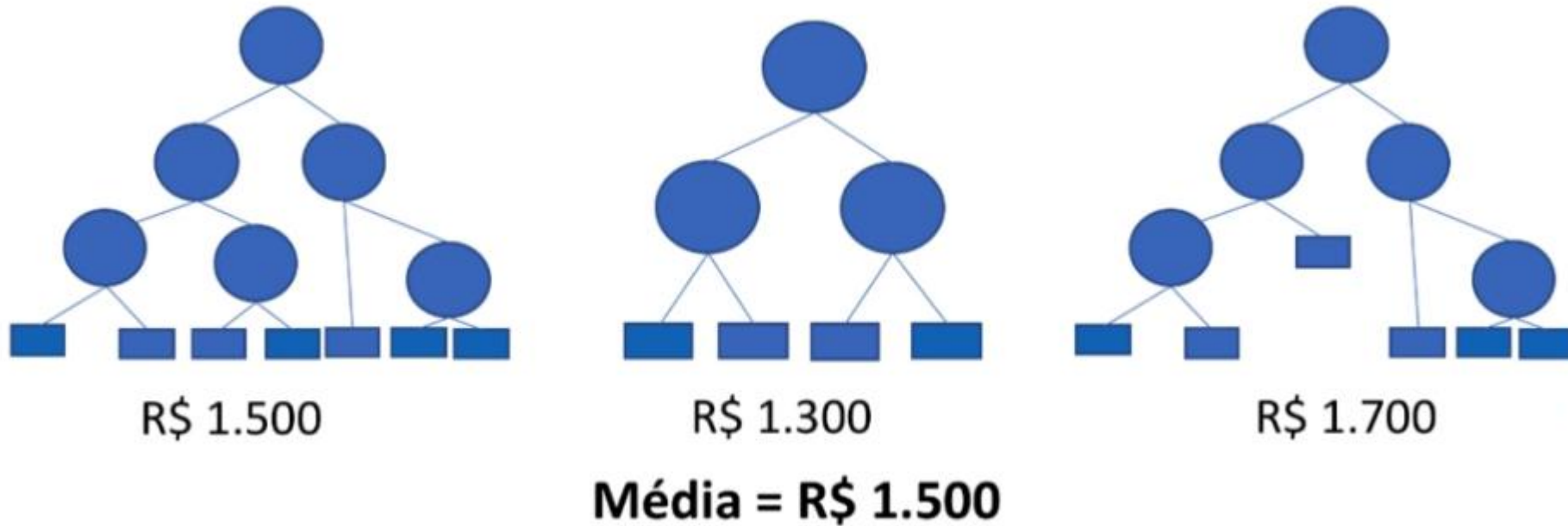
- Em problemas de regressão, a resposta final será a média da resposta de cada árvore da floresta.
- Por exemplo: qual é a previsão de salário para a pessoa X?



Regressão com Random Forest

- **Descrição da técnica**

- Em problemas de regressão, a resposta final será a média da resposta de cada árvore da floresta.
- Por exemplo: qual é a previsão de salário para a pessoa X?



- **Construindo o script**

- Voltando ao problema dos preços de imóveis, o pré-processamento fica igual.

```
import pandas as pd

##### #Pré-processamento dos dados #####

#Leitura dos dados
base = pd.read_csv('house_prices.csv')
base.describe()

# =====
#                               Tratando valores inválidos
# =====
# Não há valores inconsistentes

# Procurando as colunas que possuem algum valor faltante
pd.isnull(base).any()

# =====
#                               Separando dados em previsores e objetivo
# =====

cols_previsores = ['bedrooms', 'bathrooms', 'sqft_living', 'sqft_lot',
                   'floors', 'waterfront', 'view', 'condition', 'grade', 'sqft_above',
                   'sqft_basement', 'yr_built', 'yr_renovated', 'zipcode', 'lat', 'long']
# Não usei: date, sqft_living15 e sqft_lot15

cols_objetivo = ['price']
previsores = base[cols_previsores]
objetivo = base[cols_objetivo]

# =====
#                               Transformar as variáveis categóricas em valores numéricos
# =====
# Todas as variáveis são numéricas

# =====
#                               Separando em base de testes e treinamento
# =====

# usando 25% para teste
from sklearn.model_selection import train_test_split
previsores_treinamento, previsores_teste, objetivo_treinamento, objetivo_teste = train_test_split(previsores,
                                                                                                  objetivo,
                                                                                                  test_size=0.25,
                                                                                                  random_state=0)

# =====
#                               Padronização dos dados
# =====

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
previsores_treinamento = scaler.fit_transform(previsores_treinamento)
previsores_teste = scaler.transform(previsores_teste)
```

- **Construindo o script**

- Para aplicar a regressão usando Random Forest, escreva o seguinte código:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics
import numpy as np

##### Preprocessamento #####

# Arquivo separado

##### Regressão com Random Forest #####

from sklearn.ensemble import RandomForestRegressor
regressor = RandomForestRegressor(n_estimators=100,      # número de árvores
                                max_features=5,      # qtd de características
                                max_depth=15,
                                random_state=0)

# Treinamento
regressor.fit(previsores_treinamento, objetivo_treinamento)

# Teste
previsoes = regressor.predict(previsores_teste)

##### Avaliação dos resultados #####

score = regressor.score(previsores_teste, objetivo_teste)
mae = metrics.mean_absolute_error(objetivo_teste, previsoes)
mse = metrics.mean_squared_error(objetivo_teste, previsoes)
rmse = np.sqrt(metrics.mean_squared_error(objetivo_teste, previsoes))

print('Score:', score)
print('Mean Absolute Error:', mae)
print('Mean Squared Error:', mse)
print('Root Mean Squared Error:', rmse)
```

- **Construindo**

- Para aplicar o código:

Vamos registrar os resultados em um arquivo único, para que possamos compará-lo com resultados futuros:

```
Score: 0.888860523817985  
Mean Absolute Error: 67829.23852500941  
Mean Squared Error: 14763833351.63137  
Root Mean Squared Error: 121506.51567562691
```

o seguinte

```
##### Preprocessamento #####  
  
# Arquivo separado
```

House Sales in King County, USA				
Descrição	Dados e imóveis e seus respectivos valores em um distrito dos Estados Unidos.			
Total de registros:	21613			
Total de características:	16			
Total de colunas objetivo:	1			
Resultados				
Método	Configuração	Tratamento de dados	Score %	RMSE
Regressão Linear	-	Padronização	0.69	202633.35
Regressão Polinomial	Grau 2	Padronização	0.8083	159537.82
Árvores de Decisão	max_depth=9	Padronização	0.8002	162896.46
Random Forest	n_estimators=100, max_features=10, max_depth=15	Padronização	0.8889	121506.52

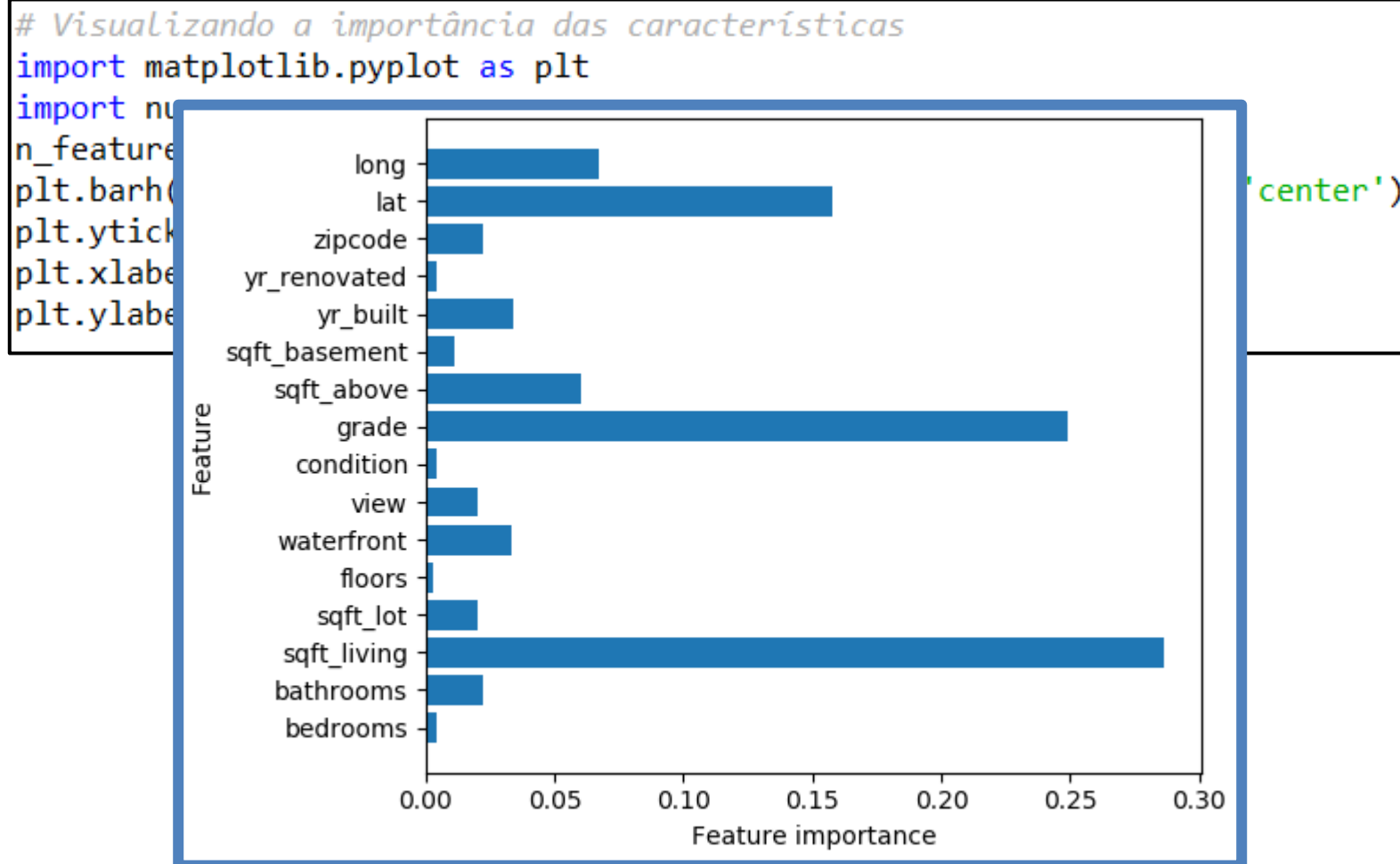
- **Construindo o script**

- Também é interessante gerar um gráfico com a importância que cada característica da base teve no resultado final.
- Para isso, acrescente as seguintes linhas ao final do script:

```
# Visualizando a importância das características
import matplotlib.pyplot as plt
import numpy as np
n_features = previsores.columns.size
plt.barh(range(n_features), regressor.feature_importances_, align='center')
plt.yticks(np.arange(n_features), previsores.columns)
plt.xlabel("Feature importance")
plt.ylabel("Feature")
```

- **Construindo o script**

- Também é interessante gerar um gráfico com a importância que cada característica da base teve no resultado final.
- Para isso, acrescente as seguintes linhas ao final do script:



Regressão com Árvores de Decisão

FIM